Track 2: AI Data Center Power–Computing Co-Scheduling

Caitlin Zhou, Amy Li, Sunny Jin

1 Summary

The HackSHSID Track 2 problem ponders how an AI data center should coordinate computing work and electricity purchases when green power supply, grid prices, and task arrivals all fluctuate over a 24-hour horizon. This paper formulates the co-scheduling problem as a mixed-integer linear program (MILP) that decides, for every hour of the day, how much of each task to execute and how to power it from green sources, the traditional grid, and a battery energy storage system (BESS). The model is solved to proven optimality with branch-and-bound and is benchmarked against a greedy, urgency-first baseline that runs on identical inputs.

The optimized schedule reduces operating cost by 15.3% (from ¥41,773 to ¥35,374) and carbon emissions by 29.7% (from 7,145 kg to 5,019 kg CO_2) while completing 100% of tasks within their deadlines. Sensitivity analysis across five dimensions — the carbon price, green power scarcity, battery capacity, grid carbon intensity, and the urgency composition of the workload — shows that the advantage is structural: it persists under every tested condition. The analysis also identifies where the leverage disappears (a fully green-scarce day roughly doubles cost and increases emissions six-fold) and reveals that under the current price profile the cheapest schedule is automatically the cleanest, so the monetary and environmental objectives never conflict.

2 Assumptions and Justifications

Assumption: One data center in one specific geographic location is optimized.
Justification: As the problem prompt addresses only one AI data center, and simultaneously monitoring emission and price fluctuations across states or national borders is unfeasible, we focus on one data center instead of spatially delegating tasks.

Assumption: We consider 4 types of tasks, based on their urgency and power demand:

- **Latency-Critical (Interactive) Workloads**: High Urgency
- **Best-Effort (Batch) Workloads**: Medium Urgency
- **AI/Large-scale Training**:Low Urgency
- **Maintenance**: Considered in power to computation efficiency

Justification: The classification of workloads into Interactive and Batch originates from Google's Borg architecture, a system for cluster management[1]. It has been deployed internally since 2003-2004, distributing heterogeneous tasks across data centers, and thus provides a reliable classification for our purpose. The background operation of data centers implicates Maintenance, while AI/Large-scale Training workloads stem from the development of enhanced deep-learning models.

Assumption: All tasks must be completed by their deadlines.
Justification: In reality, data centers must stay on top of all tasks commissioned by clients; missing deadlines detriments their operation and reputation greatly, so they are prohibited in our model.

Assumption: Three types of energy sources may fuel data center operations:

- Green or Renewable Energy
- Traditional Grid Energy
- Energy stored and discharged via a Battery Energy Storage System (BESS)

Justification: Implementing green energy as an umbrella term allows for flexibility across varying geographic regions, while differences between different renewable sources are minimal. Additionally, most modern data centers house a BESS for efficiency, including the sponsors of this problem, Huawei, which utilizes lithium-ion batteries.

Assumption: The battery must end the planning horizon at least as full as it started, i.e. $B(H) \geq B_{\text{init}}$.
Justification: Without this condition, the optimizer treats the initial state of charge as free energy: it drains the battery for profit and never refills it, which no real operator would allow. Requiring the day to end with the starting reserve makes the optimization honest and keeps the comparison against the baseline fair.

3 Constrained Multi-Objective Optimization

3.1 Approach

The co-scheduling problem is formulated as a constrained multi-objective optimization over a 24-hour horizon discretized into hourly time steps ($H = 24$, $\Delta t = 1$ h). Hourly discretization matches the timescale at which the exogenous signals change — time-of-use tariffs and green supply profiles are quoted per hour — and at which market-level energy decisions are made. Sub-hourly sequencing of jobs is delegated to the cluster management layer beneath this model; the model decides how much capacity and energy each hour receives.

The complete model consists of the decision variables in Table 5, the constraints of Section 3.3, and the objective of Section 3.4. The instance studied here contains $J = 52$ tasks (14 high-urgency, 24 medium-urgency, 14 low-urgency), 24 task-completion constraints, and 24 binary battery-mode variables. The model is expressed with the PuLP modeling library and solved with the HiGHS branch-and-bound solver, which *certifies* the optimality of the returned schedule: the solution is accompanied by a proof that no feasible schedule achieves a lower objective value, rather than merely appearing to work. The same formulation admits CBC or any standard MILP solver without modification.

Because an optimizer's value is only meaningful relative to a naive alternative, the model is benchmarked against a greedy baseline (Section 4.1) on an identical task set, price profile, and green supply. Both schemes are then scored by the same five metrics: total monetary cost (¥), total carbon emissions (kg CO_2), service-level attainment (SLA: the fraction of tasks fully completed by their deadlines), green power utilization (the share of available green energy actually used), and peak grid draw (the largest single-hour grid purchase, to which demand charges are proportional).

3.2 Defined Parameters

3.2.1 Set-Up

Table 1: Temporal Variables

Symbol	Variable Description
$t \in \{0, 1, \ldots, H-1\}$	Set and Indices of Time Steps
Δt	Length of One Time Step
H	Planning Horizon

Table 2: Task Variables

Symbol	Variable Description
$j \in \{1, 2, \ldots, J\}$	Set and Indices of Tasks
a_j	Arrival Time Step of Task j
d_j	Deadline Time Step of Task j; this varies based on the urgency class of task j: • High: $d_j = a_j + 1$ • Medium: $d_j = a_j + 4$ • Low: $d_j = H$
w_j	Total Work Required by Task j, in kWh
c_j	Urgency Class of Task j, $c \in$ {High, Medium, Low}

Table 3: Power Supply Variables

Symbol	Variable Description
$G(t)$	Green Power available/time step (kWh/step)
π_g	Green Power Price (¥)
π_t	Traditional Grid Power Price (¥)
γ_g	Green Power Carbon Intensity (0.02 kg CO_2/kWh, consolidating life cycle carbon intensities of multiple green power sources)
γ_t	Traditional Grid Power Carbon Intensity (0.394 kg CO_2/kWh, according to the United States Environmental Protection Agency [2])

Table 4: Data Center Capacity

Symbol	Variable Description
C	Maximum compute power the data center can process per step (kWh), defined to be 10000, as Data Center Asia suggests [3].
η	Power-to-compute efficiency (work output per kWh input), defined to be 0.67, citing the average Power Usage Effectiveness Google lists [4].

3.2.2 Decision Variables

Table 5: Decision Variables

Variable	Domain	Description
$x_j(t)$	≥ 0	Work done on task j during time step t (kWh)
$P_g(t)$	≥ 0	Green power consumed during step t (kWh)
$P_t(t)$	≥ 0	Traditional grid power consumed during step t (kWh)
$P_{\text{ch}}(t)$	≥ 0	Battery charge power at step t (kWh)
$P_{\text{dis}}(t)$	≥ 0	Battery discharge power at step t (kWh)
$B(t)$	≥ 0	Battery state of charge at start of step t (kWh)

3.3 Constraints

$$\sum_{t=a_j}^{d_j} x_j(t) = w_j, \quad \forall j \tag{1}$$

Explanation: Referencing our assumption, every task must be fully completed by its deadline.

$$\frac{\sum_j x_j(t)}{\eta} \leq C \tag{2}$$

Explanation: The total work scheduled per step cannot exceed data center capacity.

$$\frac{\sum_j x_j(t)}{\eta} + P_{\text{ch}}(t) = P_g(t) + P_t(t) + P_{\text{dis}}(t) \quad \forall t \tag{3}$$

Explanation: To achieve a power balance, the total power consumed equals the total power supplied at any given time step.

$$P_g(t) \leq G(t) \quad \forall t \tag{4}$$

Explanation: No more green power can be used than is available.

$$\begin{aligned} B(t+1) &= B(t) + \eta_{\text{ch}} \cdot P_{\text{ch}}(t) \cdot \Delta t - \frac{1}{\eta_{\text{dis}}} \cdot P_{\text{dis}}(t) \cdot \Delta t \quad \text{for } t = 0, \ldots, H-2 \\ B(0) &= B_{\text{init}} \end{aligned} \tag{5}$$

Explanation: The state-of-charge evolution of batteries.

$$\begin{aligned} 0 \leq B(t) \leq B_{\max} \quad \forall t \\ 0 \leq P_{\text{ch}}(t) \leq P_{\text{batt,max}} \times z(t), \quad z(t) \in \{0,1\} \\ 0 \leq P_{\text{dis}}(t) \leq P_{\text{batt,max}} \times (1 - z(t)) \\ B(H) \geq B_{\text{init}} \end{aligned} \tag{6}$$

Explanation: Bounds of battery parameters. The final line enforces the end-of-horizon reserve (Assumption 5): the battery may not end the day below its initial state of charge, preventing the optimizer from treating the initial reserve as free energy.

$$x_j(t) \geq 0, P_g(t) \geq 0, P_t(t) \geq 0, P_{\text{ch}}(t) \geq 0, P_{\text{dis}}(t) \geq 0, B(t) \geq 0 \tag{7}$$

Explanation: The non-negativity of the above parameters.

3.4 Objective Function

Cost Term:

$$\text{Cost} = \sum_{t=0}^{H-1} [\pi_g(t) \cdot P_g(t) + \pi_t(t) \cdot P_t(t)] \cdot \Delta t \tag{8}$$

The monetary cost of operation is simply computed through the prices and usage durations of different energy sources.

Carbon Term:

$$\text{Carbon} = \sum_{t=0}^{H-1} [\gamma_g \cdot P_g(t) + \gamma_t \cdot P_t(t)] \cdot \Delta t \tag{9}$$

Similarly, we derive the carbon intensity of a schedule.

Final Objective Function:

$$Z = \alpha \cdot \text{Cost} + \beta \cdot \text{Carbon} \tag{10}$$

Finally, the function we seek to minimize is simply the cumulative "costs", monetary and environmental. Note that since normalization for both terms would be unviable, we convert the carbon intensity to ¥, referring to the Social Cost of Carbon, $ 283 per ton, calculated by UC Davis scholars [5]. This yields a carbon weight of $\beta = 0.283$ ¥/kg CO_2; the weight is treated as a policy dial in Section 4.2.

4 Model Evaluation

4.1 Comparison with Greedy Baseline

To answer "compared with a simple approach without scheduling, where is your method better, and by how much?", the MILP is benchmarked against a greedy controller that mimics a data center operated without scheduling intelligence. At every hour t, the greedy controller executes waiting tasks in urgency order (high, then medium, then low; ties broken by earliest deadline) until the hourly compute capacity $C \cdot \eta$ is saturated. It then purchases the cheapest power available at that instant: green power when $\pi_g(t) < \pi_t(t)$, otherwise grid power. It never defers work to a future hour and never operates the battery, since both behaviors require anticipation; the battery therefore remains at its initial state of charge and satisfies the end-of-horizon reserve trivially. Both schemes face the same 52-task set, the same price profiles, and the same green supply.

Table 6: MILP versus greedy baseline on the base instance.

Scheme	**Cost (¥)**	**Carbon (kg CO_2)**	**SLA (%)**	**Green Util. (%)**	**Peak Grid (kW)**
MILP (optimal)	**35,373.57**	**5,019.28**	100	**80.3**	7,806.4
Greedy	41,773.13	7,144.86	100	73.7	**2,873.1**
Improvement	15.3%	29.7%	—	+6.6 pts	−171.6%

As Table 6 shows, the optimal schedule is 15.3% cheaper and 29.7% cleaner than the greedy baseline at identical SLA. The gain comes from two mechanisms the greedy controller cannot access:

1. **Load shifting.** Low-urgency tasks carry no intrinsic urgency — only a deadline of hour 23. The greedy controller executes each task the moment it arrives, frequently at off-peak green hours but sometimes at hours when only expensive grid power remains. The MILP instead parks this work until hours when cheap, clean energy is abundant. Figure 2 makes this visible: low-urgency task L1 arrives at midnight yet executes at hour 10, riding the midday green wave; tasks L9–L14 deliberately defer to hours 22–23.

2. **Battery arbitrage.** The greedy controller leaves the battery untouched. The MILP charges it from surplus green power at hours 11–12 (at zero marginal carbon), holds the energy through the afternoon, and discharges during the evening peak (hours 16–17 and 20–21) when green supply collapses and grid prices reach ¥1.1–1.3/kWh. The bottom panel of Figure 2 shows the discharge bars coinciding exactly with the expensive hours.

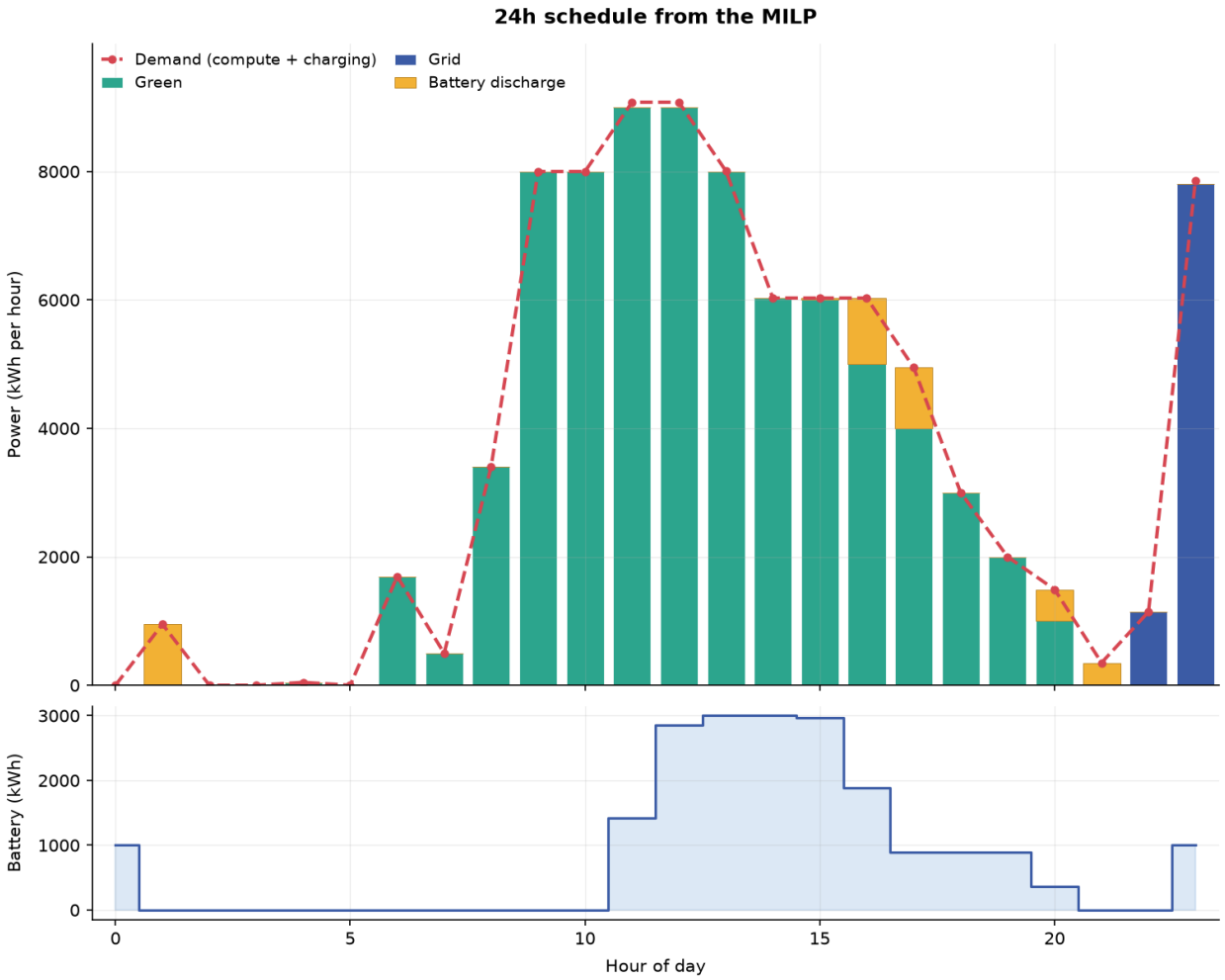


Figure 1: Optimal 24-hour schedule produced by the MILP: hourly power sources (green, grid, battery discharge), the total demand line including battery charging, and the battery state of charge. Battery discharge (amber) coincides with the evening grid-price peak.

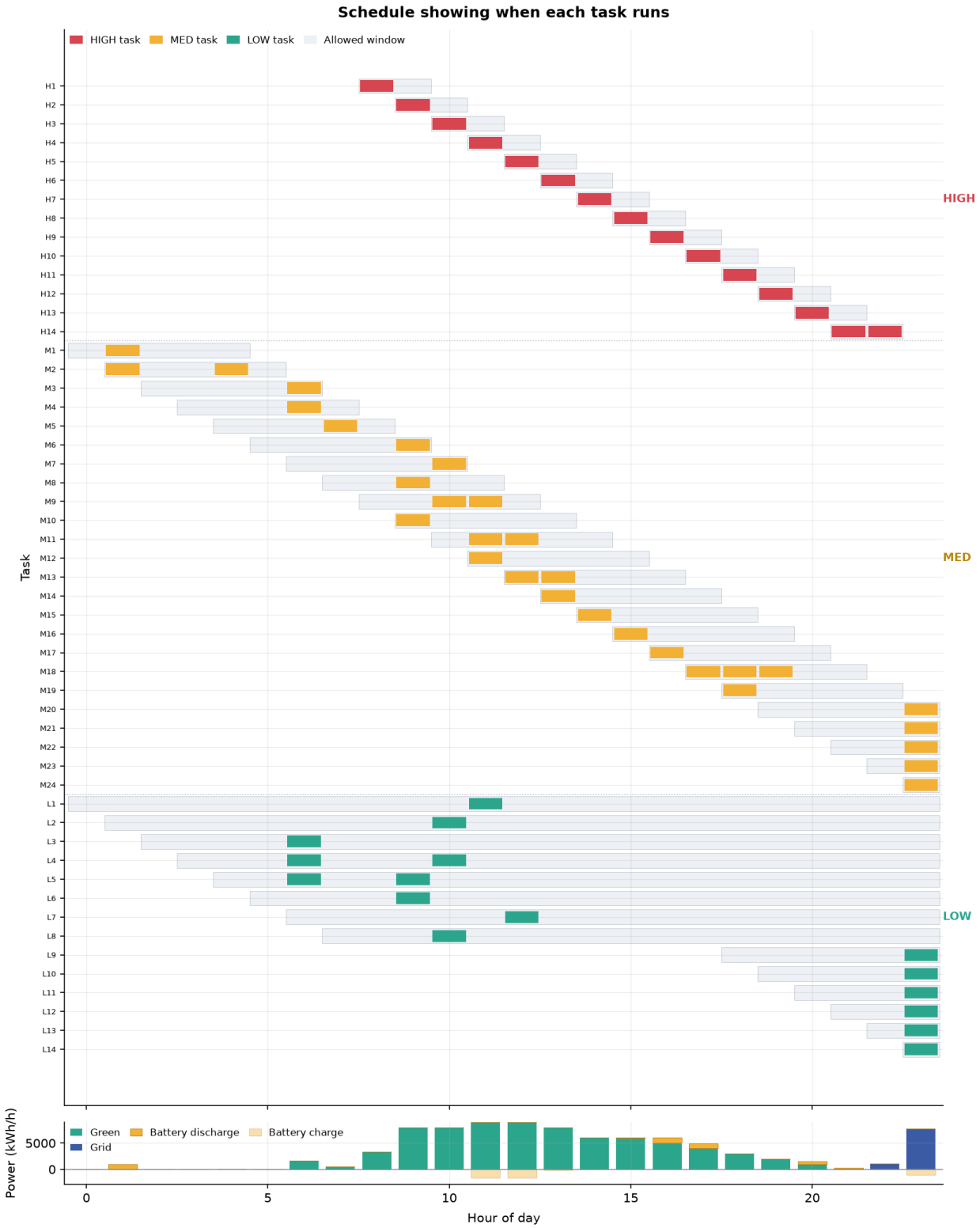


Figure 2: Schedule showing when each task runs. Pale bars mark each task's allowed window (arrival to deadline); solid bars mark executed work, colored by urgency class. The aligned bottom panel shows the power sources in the same hours. Load shifting is visible in the low urgency block: tasks wait for midday green power despite earlier arrivals.

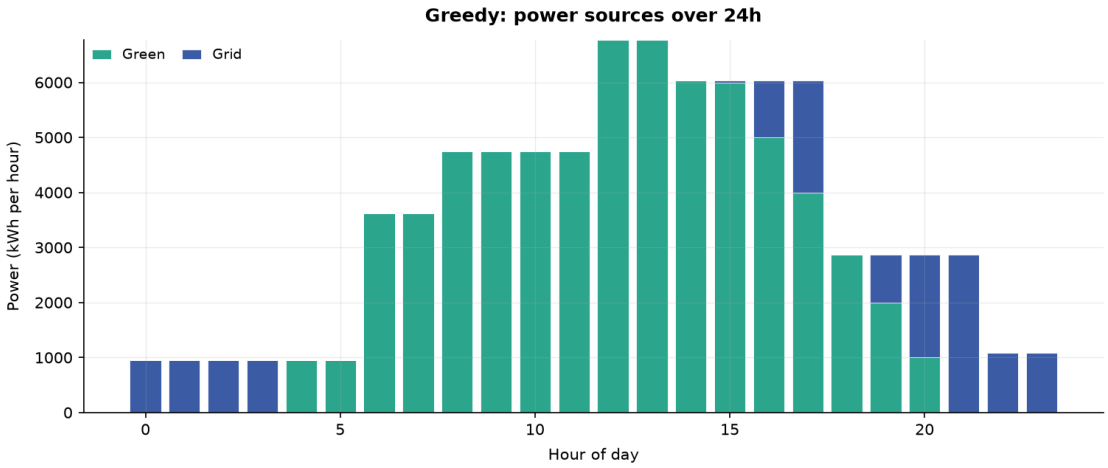


Figure 3: The greedy baseline over the same 24 hours. Without load shifting or the battery, the schedule follows the task arrival pattern directly: grid purchases (blue) cover every hour without sufficient green supply, including the expensive evening peak.

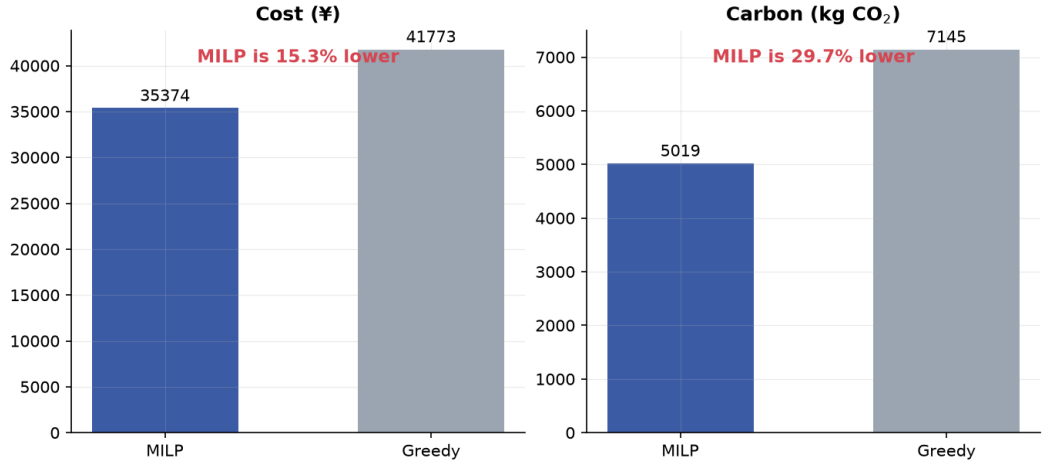


Figure 4: Total cost and carbon emissions, MILP versus greedy baseline.

One metric favors the greedy controller, and reporting it is deliberate: the optimal schedule's peak grid draw (7,806 kW) exceeds the greedy baseline's (2,873 kW), because the optimized plan concentrates grid purchases into a small number of hours — including the hour-23 battery recharge required by the end-of-horizon reserve. An operator facing steep demand charges can add a peak-limiting constraint ($P_t(t) \leq P_t^{\max}$), which the formulation accommodates directly; the trade-off between peak smoothness and cost is therefore a modeling choice rather than a structural weakness.

4.2 Sensitivity Analysis

A schedule that only works under one set of conditions is a curiosity rather than a method. Each experiment below re-solves the full MILP with exactly one parameter varied while all others remain at base values; the urgency-mix experiment additionally recomputes the greedy baseline, since it changes the task set itself. Complete numerical results appear in Table 7 and are visualized in Figures 5–9.

Table 7: Sensitivity analysis: full numerical results. Base values are printed in bold.

Experiment	Setting	Cost (¥)	Carbon (kg CO_2)
Carbon weight β	$\beta = 0$	35,374	5,036
	$\beta = 0.25$ to 2.0	35,374	5,019
Green supply scale	$G \times 0$	78,623	33,021
	$G \times 0.5$	49,245	15,515
	$G \times 1.0$ **(base)**	**35,374**	**5,019**
	$G \times 1.5$	34,244	3,357
Battery capacity $B_{\max}$	0 kWh	36,547	5,683
	1,000 kWh	36,175	5,726
	3,000 kWh **(base)**	**35,374**	**5,019**
	6,000 kWh	34,637	3,919
	10,000 kWh	34,041	2,506
Grid carbon intensity γ_t	0.2	35,374	3,283
	0.394 **(base)**	**35,374**	**5,019**
	0.57	35,374	6,594
	0.8	35,374	8,653
Urgency mix (HIGH share)	20% (MILP)	19,440	1,986
	20% (greedy)	23,082	4,105
	40% (MILP)	26,449	2,819
	40% (greedy)	30,966	5,000
	60% (MILP)	44,987	7,877
	60% (greedy)	51,948	9,955

4.2.1 Cost–Carbon Trade-off (β sweep)

Varying the carbon weight β from 0 to 2 traces the Pareto frontier between monetary and environmental objectives (Figure 5). The frontier collapses to a single point: total cost remains ¥35,374 for every value of β. Under the base price profile, green power is both cheaper and cleaner than grid power in every hour in which it is available, so the cost-minimal schedule already maximizes green utilization; the two objectives never pull against each other. The only observable effect of β is tie-breaking among equally cheap optima: with $\beta = 0$ the solver returns an alternative optimum with identical cost but slightly higher emissions (5,036 vs. 5,019 kg), while any $\beta \geq 0.25$ selects the cleaner schedule.

This degeneracy is itself a finding rather than a failure: under current prices, carbon pricing gives the optimizer no additional lever, and "cheaper" automatically means "greener." A genuine trade-off would emerge only if green power became more expensive than grid power in some hours — for instance under a power purchase agreement premium or scarcity pricing — in which case the formulation would trace a non-trivial frontier without modification.

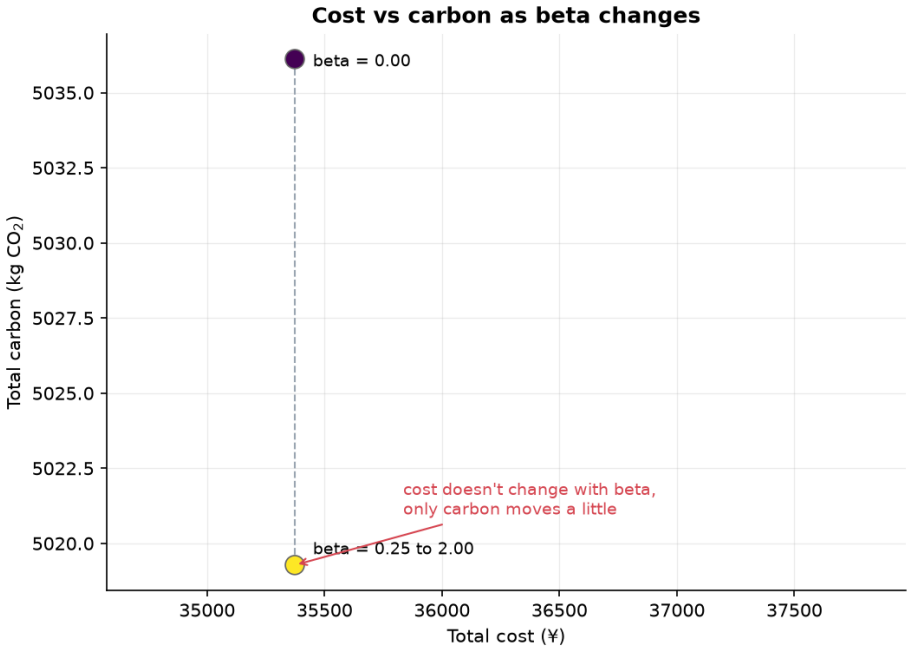


Figure 5: Cost versus carbon as the carbon weight β varies from 0 to 2. The frontier is degenerate: the cost-optimal schedule is also carbon-optimal under the base price profile.

4.2.2 Green Power Scarcity

Scaling the green supply profile $G(t)$ probes robustness to the weather (Figure 6). With zero green power, cost rises to ¥78,623 (2.2× base) and carbon to 33,021 kg (6.6× base) — the schedule degenerates to pure grid operation, as it must. At half supply, the optimizer still consumes *every* available green kWh (100% utilization) and cuts cost by 37% relative to the no-green case. Beyond the base profile the returns flatten sharply: a further 50% of green supply saves only ¥1,130, because the binding constraint shifts from power supply to compute capacity, and utilization drops to 57% — the surplus is curtailed. The schedule degrades gracefully with scarcity and cannot exploit oversupply, exactly as the physics dictates.

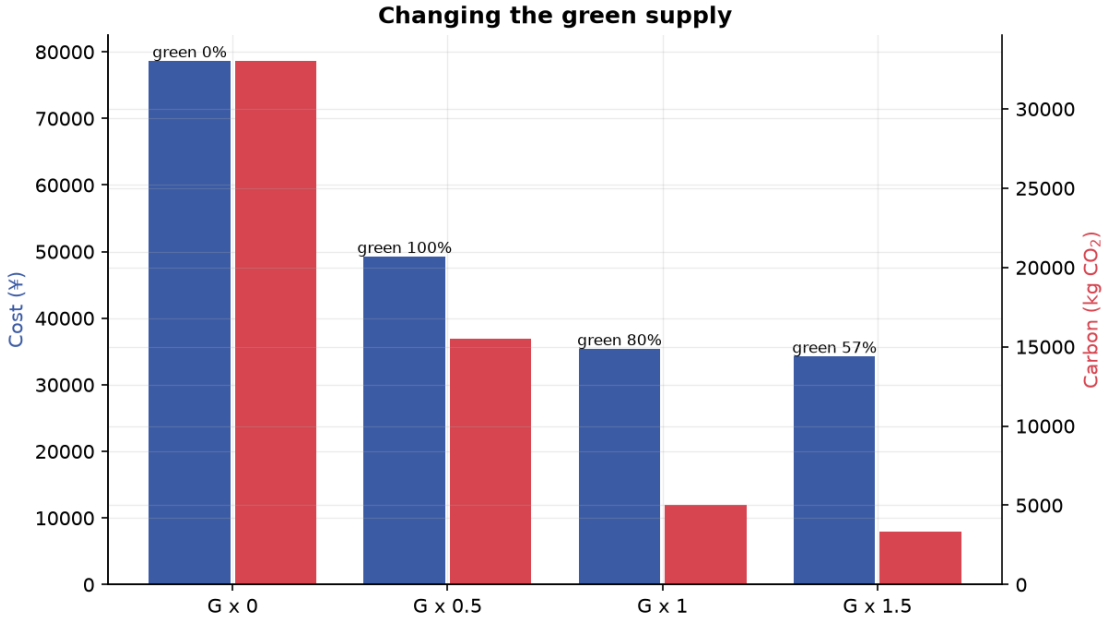


Figure 6: Green supply scaled by 0× to 1.5×. Cost (left axis) and carbon (right axis) fall steeply and then saturate; annotations show green utilization.

4.2.3 Battery Capacity

Enlarging the battery from 0 to 10,000 kWh reduces cost monotonically (¥36,547 to ¥34,041) and carbon almost monotonically (Figure 7). Two features deserve attention. First, the relationship is concave: marginal

savings decline from about ¥0.37 per added kWh in the first 1,000 kWh to about ¥0.15 in the final 4,000, because the highest-value arbitrage opportunities are consumed first. Second, carbon is non-monotonic at small capacities: a 1,000-kWh battery emits slightly *more* carbon (5,726 kg) than no battery at all (5,683 kg), since every cycle loses roughly 10% of its energy to round-trip inefficiency and the small buffer forces grid-charging before it can unlock meaningful green arbitrage. Battery value is therefore real but non-linear — a counterintuitive result directly relevant to storage sizing decisions.

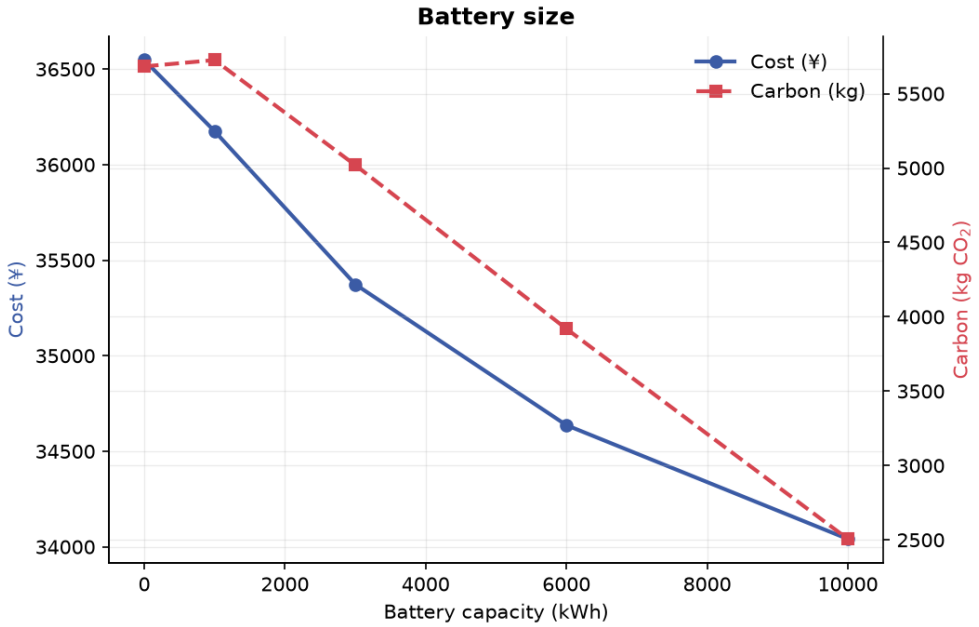


Figure 7: Cost and carbon as functions of battery capacity $B_{\max}$. Savings are concave; carbon is non-monotonic at small capacities due to round-trip losses.

4.2.4 Grid Carbon Intensity

Varying the grid carbon intensity γ_t from 0.2 to 0.8 kg CO_2/kWh leaves both the optimal cost and the entire dispatch plan unchanged (Figure 8): green power remains preferred in every hour regardless of how dirty the grid is, so total carbon simply scales linearly with γ_t (3,283 to 8,653 kg). The optimized schedule is therefore robust to uncertainty in the grid's generation mix — a misestimate of γ_t changes the reported emissions but never the plan itself.

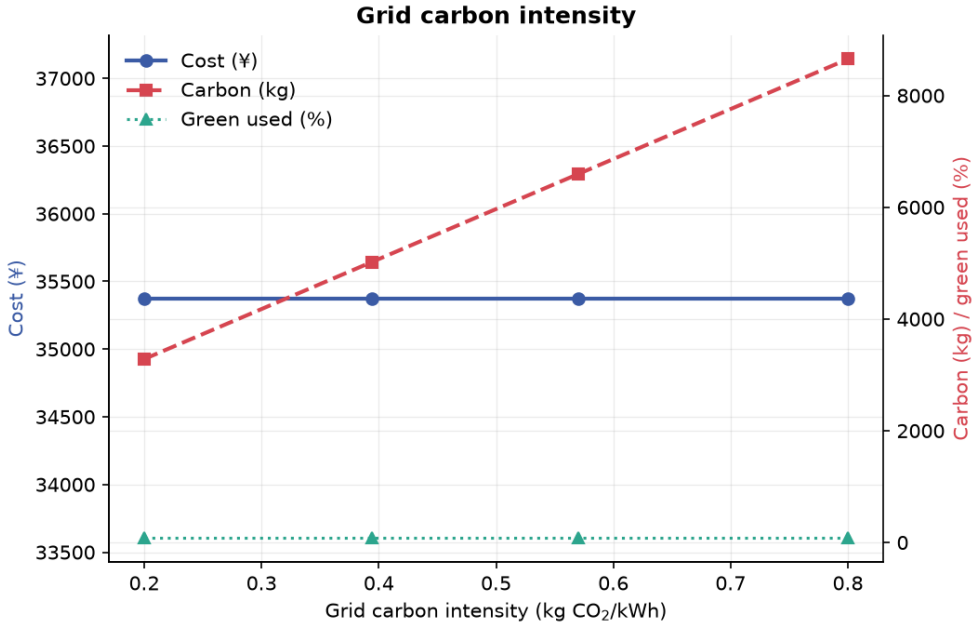


Figure 8: Grid carbon intensity sensitivity. The dispatch plan is invariant; total carbon scales linearly with γ_t while green utilization remains constant.

4.2.5 Urgency Composition of the Workload

Finally, the share of total work belonging to high-urgency (latency-critical) tasks is varied from 20% to 60% by rescaling the high-urgency workload while holding all other tasks fixed (Figure 9). Two trends emerge. More urgent work means less schedulable slack, so the optimal cost rises steeply (¥19,440 at 20% to ¥44,987 at 60%). Throughout, the MILP retains its advantage over the greedy baseline — 15.8%, 14.6%, and 13.4% cheaper at 20%, 40%, and 60% respectively — and both maintain perfect SLA. The edge narrows gently as workload becomes rigid, confirming that the value of optimization flows precisely from the flexible share of the workload.

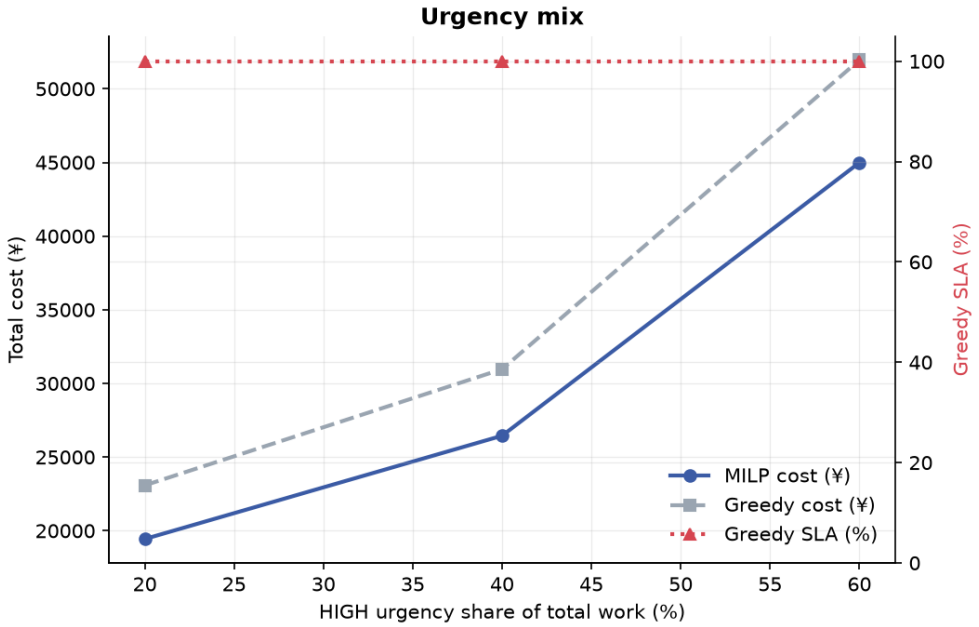


Figure 9: Urgency mix sensitivity: MILP and greedy cost (left axis) and greedy SLA (right axis) as the high-urgency share of total work varies.

5 Appendix

5.1 Code

The complete implementation is contained in the submission archive as `main.py` (Python 3, PuLP 4.x with the HiGHS/CBC branch-and-bound solvers, matplotlib for visualization). Running `python main.py` reproduces every figure and table in this paper; `python main.py --base` runs only the MILP–greedy comparison. The solver returns a certified-optimal solution for every instance; the full run completes in under one minute on a laptop.

5.2 AI Usage Statement

AI Models Used:

- Gemini 3
- Visual Studio Code Intelligence
- Visual Studio Code Copilot
- Tencent Workbuddy Kimi-K2.6

We did not appeal to AI assistance when designing our main algorithm (including the considered parameters, constraints, and objective functions), nor when completing necessary research. AI aided us in expediting code realization via autocomplete features and in suggesting code for visualization.

All AI-generated content were subjected to human supervisation and modification before being applied to our project.

References

[1] Abhishek Verma et al. *Large-scale cluster management at Google with Borg*. Proceedings of the European Conference on Computer Systems (EuroSys), Bordeaux, France. The Borg architecture distinguishes latency-sensitive (borgjob) and batch (prod/app) workloads. 2015.

[2] United States Environmental Protection Agency. *Emissions and Generation Resource Integrated Database (eGRID), Summary Tables*. `https://www.epa.gov/egrid`. U.S. grid average carbon output rate of approximately 0.394 kg CO2/kWh. 2019.

[3] Data Center Asia. *AI Data Center Power and Capacity Benchmarks*. `https://www.datacenterasia.com`. 2025.

[4] Google. *Data centers: Efficient computing*. `https://www.google.com/about/datacenters/efficiency/`. Fleet-wide power usage effectiveness statistics. 2025.

[5] Frances C. Moore et al. "The Social Cost of Carbon: A Synthesis of Estimates and Uncertainty". In: *Nature Climate Change (synthesis of U.S. Social Cost of Carbon estimates, $283 per ton CO2)* (2024).